

Signatures and Transactions

Transactions

How can we create an application/cryptocurrency on a blockchain?

- What is in the blocks?
- How to build a meaningful application from it?
- Assume anyone can submit data to the blockchain.

Digital signatures

Transactions

Digital Signatures

$$pk, sk \leftarrow \text{setup}(\kappa)$$

$$\sigma \leftarrow \text{sign}(sk, msg)$$

$$bool \leftarrow \text{verify}(\sigma, msg, pk)$$

Ideas:

- Use public key as identity.
- Put signed messages on the blockchain. $\langle msg \rangle_\sigma$
- Signed messages are called *transactions*.

Account model

Transactions

Accounts

Transactions are: $\langle pk_{from}, pk_{to}, value \rangle_\sigma$

State is: balance for each public-key

Checks:

- Is signature correct?
- Does pk_{from} have enough money?

Transactions

Accounts

Transactions are: $\langle pk_{from}, pk_{to}, value \rangle_{\sigma}$

Algorithm 1 Account transactions

```
1: balances := [pk]uint
2: for block in chain do
3:   for  $\langle pk_{from}, pk_{to}, value \rangle_{\sigma}$  in block.data do
4:     if !verify(pkto || value, pkfrom,  $\sigma$ ) then
5:       continue
6:     if balances[pkfrom] < value then
7:       continue
8:     balances[pkfrom] − = value
9:     balances[pkto] + = value
```

Transactions

Accounts

Transactions are: $\langle pk_{from}, pk_{to}, value \rangle_\sigma$

State is: balance for each public-key

Checks:

- Is signature correct?
- Does pk_{from} have enough money?

Problems:

- 1. How to deposit money?**
- 2. Replay attack!**

Transactions

Accounts

Transactions are: $\langle pk_{from}, pk_{to}, value \rangle_\sigma$

Deposit:

- Give out some money
- Deposit with someone who has money

Replay attack:

- A signed transaction can be submitted multiple times.
- Sequence numbers!

Transactions

Accounts

Algorithm 2 Account transactions

```
1: balances := [pk]uint
2: sqNrs := [pk]uint
3: for block in chain do
4:   for  $\langle pk_{from}, pk_{to}, value, sqNr \rangle_{\sigma}$  in block.data do
5:     if !verify(pkto || value || sqNr, pkfrom,  $\sigma$ ) then
6:       continue
7:     if balances[pkfrom] < value then
8:       continue
9:     if sqNrs[pkfrom] = sqNr then
10:      balances[pkfrom] − = value
11:      balances[pkto] + = value
12:      sqNrs[pkfrom] ++
```

Idea: do checks when adding transaction to chain.

UTXO model

Transactions

UTXO

UTXO: Unspent transaction output

Idea: *No balances but coins*

- For each coin store pk of owner and unique id
- Transaction spends some coins and creates new ones.

Transactions

UTXO

Transactions:

$$tx = \langle \underbrace{[(id_1, \sigma_1), (id_2, \sigma_2)]}_{\text{Inputs}}, \underbrace{[(pk_a, value_a), (pk_b, value_b)]}_{\text{Outputs}} \rangle$$

State is unspent outputs $map[id](pk, value)$

Transactions

UTXO

Transactions:

$$tx = \langle \underbrace{[(id_1, \sigma_1), (id_2, \sigma_2)]}_{\text{Inputs}}, \underbrace{[(pk_a, value_a), (pk_b, value_b)]}_{\text{Outputs}} \rangle$$

Valid if:

- Inputs refer to unspent outputs.
- Signatures are correct (with outputs public key)
- Value of all inputs larger or equal than all output values.

Transactions

UTXO

Algorithm 3 Transaction validation and maintenance of UTXO

$UTXO := \text{map}[id] \rightarrow (value, pk)$

for $tx = (inputs, outputs)$ **do**

for $(id, \sigma) \in inputs$ **do**

if $UTXO[id]$ does not exist **then**

abort

▷ invalid transaction

if $\text{verify}(tx, \sigma, UTXO[id].pk) == \text{false}$ **then**

abort

▷ invalid transaction

if sum of values of inputs < sum of values of new outputs **then**

abort

▷ invalid transaction

for $((id), \sigma) \in inputs$ **do**

$\text{remove}(UTXO[id])$

▷ output spent

for $(pk, value) \in outputs$ **do**

$UTXO[newid] = (pk, value)$

▷ add new output

Transactions

UTXO

Transactions:

$$tx = \langle \underbrace{[(id_1, \sigma_1), (id_2, \sigma_2)]}_{\text{Inputs}}, \underbrace{[(pk_a, value_a), (pk_b, value_b)]}_{\text{Outputs}} \rangle$$

- No replay attack
- What to sign: $\langle [id_1, id_2], [(pk_a, value_a), (pk_b, value_b)] \rangle$

Transactions

Accounts vs. UTXO

Accounts:

- Need nonce, also for empty account

UTXO:

- More complex
- More anonymity?
- May create dust?

Take away

A blockchain is an append only log
secured against changed.

Transactions/state changes are recorded in the blockchain.

Application state can be recreated by applying all transactions.

UTXO scripts

UTXO scripts

Spending conditions

Transactions:

$$tx = \langle \underbrace{[(id_1, rd_1), (id_2, rd_2)]}_{\text{Inputs}}, \underbrace{[(s_a, value_a), (s_b, value_b)]}_{\text{Outputs}} \rangle$$

- s_a a **spending condition**: output can be used if a value is supplied, that evaluates s_a to `true`
- rd_1 a **redeeming argument**:
should ensure the script s_{id_1} returns `true`

UTXO scripts

Pay to public key hash

script (spending condition):

input (redeeming argument):

OP_DUP
OP_HASH160
<H(pubkey)>
OP_EQUALVERIFY
OP_CHECKSIG

<signature>
<pubkey>

UTXO scripts

Pay to public key hash

Concatenate:

<signature>
<pubkey>

OP_DUP
OP_HASH160
<H(pubkey)>
OP_EQUALVERIFY
OP_CHECKSIG

UTXO scripts

Pay to public key hash

Evaluate one by one:

Stack

<signature>

<pubkey>

OP_DUP

OP_HASH160

<H(pubkey)>

OP_EQUALVERIFY

OP_CHECKSIG

UTXO scripts

Pay to public key hash

Evaluate one by one:

Stack

<signature>

<pubkey>

OP_DUP

OP_HASH160

<H(pubkey)>

OP_EQUALVERIFY

OP_CHECKSIG

<signature>

UTXO scripts

Pay to public key hash

Evaluate one by one:

Stack

<pubkey>

OP_DUP

OP_HASH160

<H(pubkey)>

OP_EQUALVERIFY

OP_CHECKSIG

<pubkey>

<signature>

UTXO scripts

Pay to public key hash

Evaluate one by one:

Stack

OP_DUP

OP_HASH160

<H(pubkey)>

OP_EQUALVERIFY

OP_CHECKSIG

<pubkey>

<pubkey>

<signature>

UTXO scripts

Pay to public key hash

Evaluate one by one:

Stack

OP_HASH160

<H(pubkey)>

OP_EQUALVERIFY

OP_CHECKSIG

<H(pubkey)>

<pubkey>

<signature>

UTXO scripts

Pay to public key hash

Evaluate one by one:

Stack

<H(pubkey)>
OP_EQUALVERIFY
OP_CHECKSIG

<H(pubkey)>
<H(pubkey)>
<pubkey>
<signature>

UTXO scripts

Pay to public key hash

Evaluate one by one:

Stack

OP_EQUALVERIFY
OP_CHECKSIG

<H(pubkey)>
<H(pubkey)>
<pubkey>
<signature>

return false?

UTXO scripts

Pay to public key hash

Evaluate one by one:

Stack

OP_CHECKSIG

<pubkey>
<signature>

UTXO scripts

Pay to public key hash

Evaluate one by one:

Stack

OP_CHECKSIG

true

UTXO scripts

Pay to public key hash

Pay to public key hash

- Smaller output
- Larger input

Pay to public key

- Smaller input
- Larger output

Outputs need to be kept in memory
UTXO, to process new blocks.

Inputs, once processed, stored only for
reproducing history.

UTXO scripts

Examples

Name	Spending condition	Redeeming argument	
P2Pk Pay to public key	Public key	Signature	
P2PkH Pay to public key hash	Hash of Public key	Public key and signature	
Multisig	m public keys and parameter k	k signatures	

Light clients and issuing transactions

Light clients

Full bitcoin node:

- Stores UTXO
- Processes all transactions
- Recommended:
 - 750GB storage
 - 2GB RAM

Light node

Simplified Payment Verification

- Process headers (check PoW)
- Ask one full node for transactions affecting your private keys
- Verify merkle inclusion proofs

Light clients

Wallet modes

	Light client	API client
Mode	Download headers	Ask full node
Verification	Verify inclusion	No verification
Risk	may omit transactions	may create fake transactions

Sending transactions

- Decide what outputs to use
- Create outputs
- Sign
- Send to the network and wait for confirmation (inclusion in block)

Optimisation problem

Sending transactions

Wallet modes

	Offline wallet	Online with your own keys	Custodial wallet
Mode	Keys stored offline	Keys accessible	Someone else controls keys
Risk	Implementation flaws?	Hackers	Hackers and fraud